Group - 13
Computer Architecture
Mid Evaluation Report
The MoE Memory Wall - Predicting & Prefetching Experts for Faster LLM Inference
GitHub Link - https://github.com/Mohit14032007/CA_Project

1) DUPLEX REPRODUCTION

Introduction & Problem Overview

Large Language Models (LLMs) based on the Mixture-of-Experts (MoE) architecture increase model capacity by replacing a conventional feed-forward network with multiple expert networks and a routing mechanism.
Instead of executing every expert for every token, an MoE model activates only a small subset (Top K) of experts selected by a router. This reduces the computation required for each token, but the complete collection of expert weights can occupy a large amount of memory.
Low Op/B can impact the performance significantly due waiting while loading from memory. Hence duplex introduces separate logic PIM functionality to exploit near memory computations to balance out the loading times.

Duplex Architecture : Logical Design

The Duplex Architecture, as per research paper 1 - splits the tasks based on the Arithmetic Intensity Scores of each task
ie based on the number of operations performed per byte of data transfer.
High Op/B tasks are performed by conventional xPU while Duplex introduces a separate Logic-PIM unit for Low Op/B tasks.

- **xPU:** Conventional compute engine for general purpose or High throughput computation
- **Logic PIM:** Processing in memory performed using compute logic on a separate HBM logic die, different from conventional dram dies only. Hence exploits memory bandwidth and near memory computation for Low Op/B operations.

A single processor architecture may not handle

1) Operations benefited through high computational throughput.
2) Operations having High data access ratio compared to computation

Hence Duplex therefore combines the conventional computational with processing in memory capabilities for different computational and memory access characteristics.

Main Architectural Components

- **GPU/xPU:** Executes conventional compute intensive operations.
- **HBM:** High Bandwidth Memory stores model weights, activations, and other data.
- **HBM Controller:** Handles conventional HBM memory requests.
- **Logic PIM controller:** Controls operations executed by Logic PIM.

- **Logic PIM GEMM Modules:** Supports Matrix multiplication operations near memory.
- **Activation Module:** Execute supported Activation Operations.
- **Softmax Module:** Supports softmax computation, including attention related operations.
- **Serving Scheduler:** Manages requests, batching, and inference stages.
- **Cluster:** Coordinates devices, distributes model components, and combines execution time information.

Duplex - LLM Request Flow as Described in Research Paper 1

1) **LLM Inference Request**
2) **Serving Scheduler** - The schedule determines which requests & tokens are processed and supply workload information to the cluster
3) **Request & Batch Management** - How much Requests in a batch
4) **Cluster Manager** - Creates the configured device components.
5) **Distribute Work to Devices**
6) **Transformer Decode Layer**
   Assign operations between GPU/Logic PIM based on their Arithmetic Intensity and then perform the operations required to calculate attention outputs.
   a) **Attention Block - Attention Operation - Operation Path**
   b) **MoE Block - Router Operation - Select Top K - Route Token Data**
      The router selects the experts that will process that particular token
      For Mixtral - 32 layers, 8 experts per layer and top 2 experts are chosen per layer per token
   c) **Select between xPU and Logic PIM**
   d) **Complete Operations & Combine required outputs**
7) **Complete Decode Layer**
8) **Continue to next Layer**
9) **Generate Next Token**

Expert Execution Flow

Each selected expert contains a feed-forward network, The expert performs transformations on the token representation using its associated weights.

1) **Token Representation**
2) **Selected Expert**
3) **Gate Projection**
   a) **Up Projection**
   b) **Gate Output**
4) **Activation**
5) **Down Projection**
6) **Expert Output Vector**

The 4 reproduced configurations as per Research Paper 1

1) A3/GPU Baseline
2) A4/Duplex - GPU + Logic PIM
3) A5/Duplex + Parallel Execution
4) A6/Duplex + PE + Expert Tensor Parallelism

Expert Weights in HBM initially

Initially as per research paper 1 the experts weights are resident in the HBM only. Hence After the MoE layer selects experts, then access the weights from HBM for computation and output generation.
Also, ET - distributes the expert computation across multiple devices.

System Configuration for A100 - Duplex

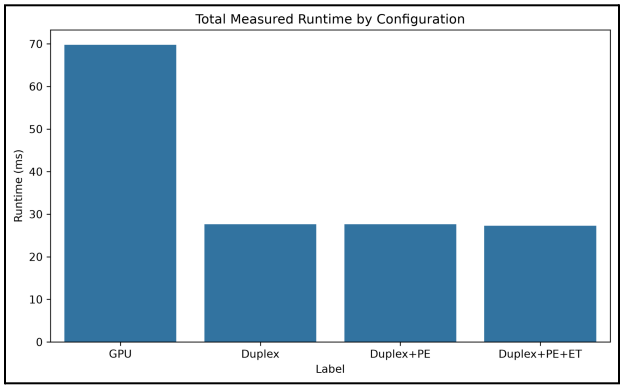
```
None
model_name: mixtral
gpu_gen: A100
processor_type: GPU + LOGIC
```

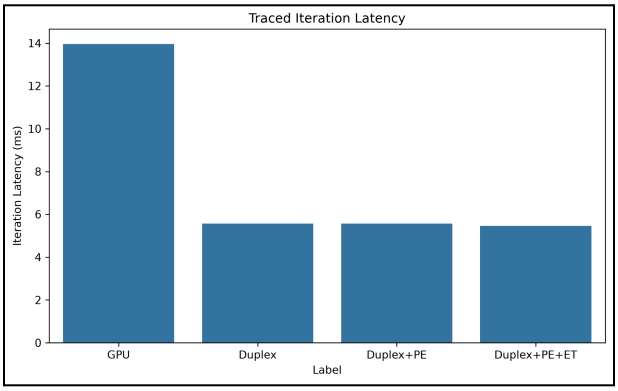
Functional Call Flow for Duplex Architecture in LLM Simulator

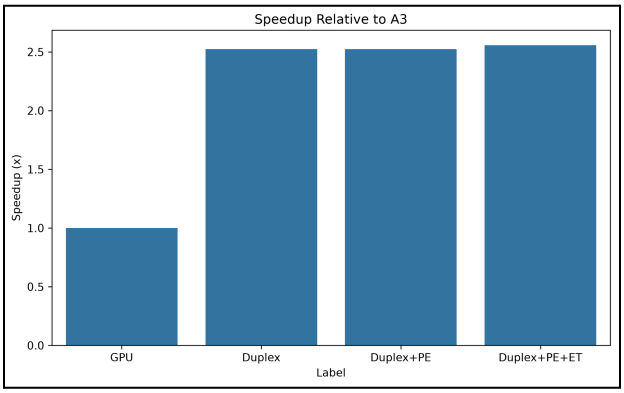
- Load **"a100_duplex.yaml"** Configuration file
- From **"src/model/model_config.h"** load Model Configurations for specified model name inside the **"a100_duplex.yaml"** Configuration file.
- Create Cluster using **"src/hardware/cluster.cpp"**
- Create devices using **"src/hardware/device.cpp"** using functions like **"Device(), run_ramulator(), run_ideal()"**
- Schedule Requests using **"src/scheduler/scheduler.cpp"**
- Use the "**Decoder::forward()"** function call from **"src/module/decoder.cpp"** to start the decode layer operations.
- Route the MoE using **"src/module/route.cpp"**
- Use **"ExpertFFN::forward()"** from **"src/module/expert.cpp"** to forward experts.
- Perform Linear Operations using **"src/hardware/linear_impl.cpp"** and use device memory path from **"src/hardware/device.cpp".**
- Use the HBM model from **"src/dram/dram_interface.cpp"** and use **"src/dram/mmap_controller.cpp"** to create mapping between logical memory requests and memory resources required.
- Simulate using Ramulator Memory and generate Timing information using **"src/module/timeboard.cpp"** and write results back using **"src/hardware/cluster.cpp"**

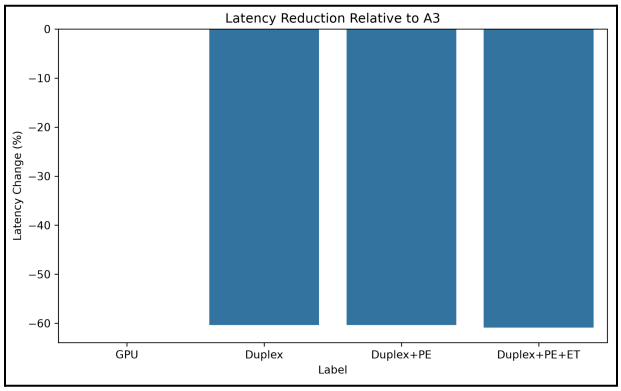
Experimental Configurations

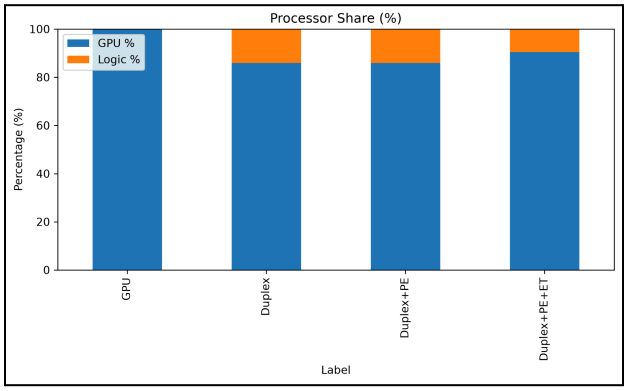
Configuration	Architecture	PE	ET	No of Device
A3	GPU Baseline	No	1	4
A4	Duplex	No	1	4
A5	Duplex + PE	Yes	1	4
A6	Duplex + PE + ET	Yes	4	4

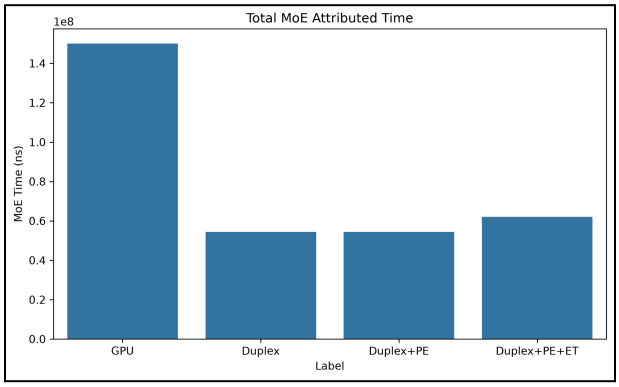


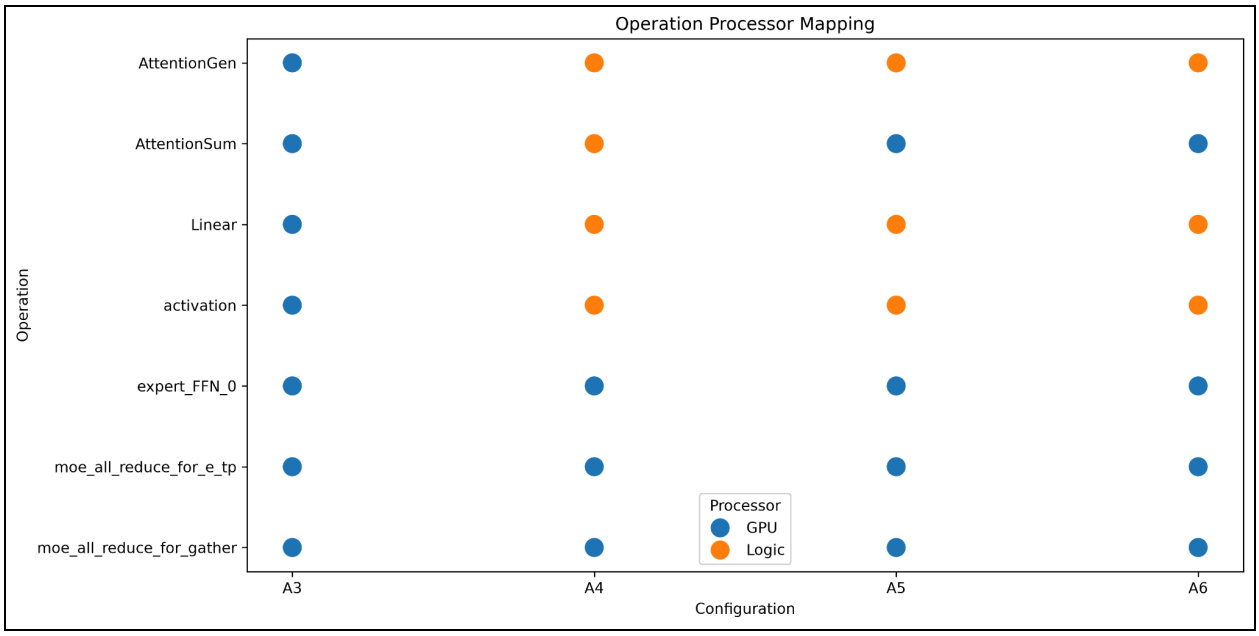












Results & Analysis (Based on Reproduced Plots)

Metric	A3	A4	A5	A6
Total Measured Runtime(ms)	69.79	27.66	27.66	27.28
Traced Iteration Latency(ms)	13.96	5.56	5.56	5.46
Speedup Relative to A3	100%	252%	252%	255%
Latency Reduction Relative to A3	0%	60.37%	60.37%	60.90%

Interpretations

- There is significant performance improvement between GPU only and GPU + Logic PIM
- The above plots show how Attention/Activation Operations are performed by Logic PIM instead of xPU/GPU and hence these shifts give significant speedup of 252% relative to GPU only.
- The resulting runtime reduction supports the conclusion that the Duplex configuration substantially improves the performance of this workload under the simulator's configured timing model.
- Percent share plots show the distribution between the GPU and Logic PIM operations & how these operation shifts caused significant latency reduction compared to A3.
- Parallel execution does not improve runtime for this simulation workload.

2) OFF CHIP DRAM
(No GPU Sharding (Device:1) + Logic PIM off + All experts weights in additional offchip dram)

Introduction & Motivation

The Duplex Reproduction Investigates the performance differences when we split the operations between xPU and Logic PIM while all expert weights reside in HBM, but that's not always the case, Hence the primary objective of this part is to investigate the memory bottleneck created when MoE model requires expert weights that are not local resident of HBM.

Hence this experiment addresses a different problem
What happens when expert weights must be transferred from an additional offchip DRAM into the HBM before the GPU can execute the corresponding expert?
Which is a genuine question to ask, as MoE models can contain a large number of experts and it is not possible for all the weights to reside in HBM due to memory constraints. This additional loading stage can increase latency and leave the GPU waiting for required weights before computation.

Research Paper 2 identifies this problem as a central motivation to incorporate spatial-temporal expert correlations to design prefetching algorithms so that we can load the data well in advance before the GPU needs it using some predictions. The paper observes that MoE workloads can exhibit correlations across adjacent MoE layers and consecutive decoding tokens. These correlations can potentially be exploited to load experts before they are needed.

Our experiment breaks down the problem statement into 5 conceptual parts

- **Expert-Weight Capacity:** The complete set of expert weights may be too large to be locally resident into HBM, hence requires an additional off chip DRAM having all expert weights. Also, the system needs a mechanism to check if an expert is currently a resident of HBM or not.
- **Expert-Loading Latency:** If the selected expert is absent from HBM. its weights must be loaded before the corresponding FFN operations can execute.
- **Residency Management:** The simulator needs to track expert residency and determine when an expert is loaded, accessed again, or evicted. Hence we need to implement an eviction policy also.
- **Expert Access Predictability:** To design a prefetching algorithm we need to observe the temporal and spatial access patterns across layers and tokens.
- **Reproducible Evaluation:** The simulator must store enough information to distinguish actual modeled memory transfers from cache hits, and to validate performance measurements.

High Level Flow

1) MoE Model
2) MoE Routing Layer
3) Expert Selection
4) Residency Check in HBM
    a) If Present, Proceed Normally
    b) If not Present - Go to Off Chip DDR5 DRAM - Using Ramulator 2 - Check Loading Time - Check HBM Available Space - Evict(LRU)/Now Present in HBM
5) Expert FFN computation
6) Expert Output & Continue Decoder

Model Configuration & Expert Weights

- Model: deepseekV3
- Number of Layers: 60 (3 Dense + 57 MoE)
- 256 experts per layer
- Top 8 selected per token per layer
- Expert FFN Structure has three principal weight matrices:
    - W1: Gate Projection
    - W2: Down Projection
    - W3: Up Projection
    - Each Matrix Contains 7168x2048
    - Size of one expert = 3 x 7168 x 2048 x 2 = 88080384 bytes = 88MB
    - Total = 57 x 256 x 88MB = 14592 x 88MB = 1.285TB

High Level Implementation Idea

The implementation combines the Mixture of Experts modified execution pipeline with LRU based expert residency management mechanism to evict experts when available size is less than the required HBM size.

The MoE router selects top-K experts required to process each token, and the corresponding ExpertFFN modules execute their gate, up and down projections using W1, W2, W3 weights identified by “expert.cpp”. Liner::forward() in linear.cpp checks whether the required expert is already a resident of HBM using the Device::is_expert_resident() function.

Already resident experts continue with the normal execution path while LRU updates their status. Non resident experts are loaded from off chip dram using sister weight loading, “linear_impl.cpp” groups sister W1,2,3 weights into a single expert level loading batch. Then “layer_impl.cpp” and “device.cpp” requests to the secondary off chip ramulator based interface. Then “Device::mark_expert_resident()” to mark expert residency into HBM. The LRU maintains a track of expertIDs, sizes, current resident byte count and order of accesses. The combined design therefore connects MoE routing, expert-weight loading, DDR5 timing simulation, HBM residency tracking, and LRU eviction into a single execution path, while the generated routing,

loading, and cache-event traces provide the information needed to analyze transfer volume, loading latency, cache hits, and evictions.

Complete MoE Execution Flow

1) The **“src/scheduler/sequence.cpp”** records which token is selected for each layer and records the whole sequence into **“expert_routing.csv”**.
2) **“src/module/expert.cpp”** constructs the expertFFN, Instead of independently loading each matrix, a whole batch containing all three matrices can be processed.
3) Expert Residency in HBM is then checked from **“src/module/linear.cpp”**

```C/C++
A->weight_target = MemoryTarget::HBM;
A->load_sisters = false;

if (A->is_expert_weight && input->getSize() > 0) {
    int layer_id = sequences_metadata->cur_layer;
    A->expert_layer_id = layer_id;

    if (!device->config.offchip_dram_cfg_path.empty()) {
        if (!device->is_expert_resident(layer_id, A->expert_id)) {
            A->weight_target = MemoryTarget::OFFCHIP_DRAM;
            A->load_sisters = true;
        }
    }
}
```

4) Then Batching of three weight tensor is done using **“src/hardware/linear_impl.cpp”** and submits the entire batch using **“ExecStatus”**

```C/C++
std::vector<Tensor_Ptr> batch_tensors;
batch_tensors.push_back(weight);

if (weight->load_sisters) {
    for (auto sister : weight->sister_weights) {
        sister->expert_layer_id = weight->expert_layer_id;
        batch_tensors.push_back(sister);
    }

    weight->load_sisters = false;
}
```

```
C/C++
ExecStatus weight_status =
    issueRamulatorBatch(
        device,
        LayerType::LINEAR,
        ProcessorType::GPU,
        DRAMRequestType::kRead,
        PIMOperandType::kDRAM,
        batch_tensors,
        weight->weight_target);
```

5) Then route the batch to correct memory interface using **“src/hardware/layer_impl.cpp”**

```
C/C++
bool bypass_cache = (target == MemoryTarget::OFFCHIP_DRAM);

device->run_ramulator_batch(requests, target);
```

The run_ramulator_batch is implemented through **“src/hardware/device.cpp”**

```
C/C++
void Device::run_ramulator_batch(
    const std::list<DRAMRequest_Ptr>& requests,
    MemoryTarget target) {

    if (target == MemoryTarget::OFFCHIP_DRAM &&
        offchip_dram_interface != nullptr) {

        offchip_dram_interface->HandleRequest(requests, 0);

    } else {

        dram_interface->HandleRequest(requests, 0);
    }
}
```

Hence the offchip dram requests are submitted to the secondary Ramulator Backed Memory interface.

6) Update the HBM residency after loading using **“src/hardware/linear_impl.cpp”**

```
C/C++
time_ns weight_memory_cost = weight_status.memory_duration;
```

```C/C++
if (weight->is_expert_weight) {
    if (weight->weight_target == MemoryTarget::OFFCHIP_DRAM) {
        long long total_expert_size = 0;

        for (auto t : batch_tensors) {
            total_expert_size += t->getSize();
        }

        device->mark_expert_resident(
            weight->expert_layer_id,
            weight->expert_id,
            total_expert_size);
    }
}
```

7) Record all the load records into **“expert_load.csv”**
8) **Execute the Expert FFN**
    a) If Expert not resident (functions used)
        i) Linear::forward()
        ii) is_expert_resident()
        iii) Make offchip request + Load + DDR5 Ramulator 2
        iv) mark_expert_resident()
        v) Record load duration and continue FFN execution
    b) If Expert Resident
        i) Linear::forward()
        ii) is_expert_resident()
        iii) Cache Hit
        iv) Use HBM target and execute

LRU Cache Implementation

- src/hardware/device.h
- src/hardware/device.cpp

Cache State

```C/C++
std::list<std::tuple<int, int, long long>> lru_expert_list;

std::unordered_map<
    std::pair<int, int>,
    std::list<std::tuple<int, int, long long>>::iterator,
    PairHash> resident_experts;

long long cache_capacity_bytes =
    std::getenv("DEBUG_CACHE_CAPACITY")
        ? std::stoll(std::getenv("DEBUG_CACHE_CAPACITY"))
        : 80000000000LL;

long long resident_bytes = 0;

//Data Structure tracks
//Expert ID, Expert Size, Current Residency, LRU Order, Total Resident Bytes,
//Configured Cache capacity
```

New Expert Addition Check

```C/C++
while (resident_bytes + expert_size > cache_capacity_bytes &&
       !lru_expert_list.empty()) {

    auto lru_expert = lru_expert_list.front();

    lru_expert_list.pop_front();

    std::pair<int, int> evict_key = {
        std::get<0>(lru_expert),
        std::get<1>(lru_expert)
    };

    resident_experts.erase(evict_key);

    resident_bytes -= std::get<2>(lru_expert);
}
```

Check expert residency

```C/C++
//src/hardware/device.cpp
bool Device::is_expert_resident(int layer_id, int expert_id) {
  static bool init = false;
  static std::ofstream csv;

  if (!init) {
    const char* trace_dir_env = std::getenv("TRACE_DIR");

    std::string path = "expert_cache.csv";

    if (trace_dir_env)
      path = std::string(trace_dir_env) + "/expert_cache.csv";

    csv.open(path, std::ios::app);
    csv.seekp(0, std::ios::end);

    if (csv.tellp() == 0) {
      csv << "layer_id,expert_id,action,resident_bytes\n";
    }

    init = true;
  }

//src/module/linear.cpp
A->weight_target = MemoryTarget::HBM;
A->load_sisters = false;

if (A->is_expert_weight && input->getSize() > 0) {
    int layer_id = sequences_metadata->cur_layer;

    A->expert_layer_id = layer_id;

    if (!device->config.offchip_dram_cfg_path.empty()) {
        if (!device->is_expert_resident(layer_id, A->expert_id)) {
            A->weight_target = MemoryTarget::OFFCHIP_DRAM;
            A->load_sisters = true;
        }
    }
}
```

Cache Hit

```C/C++
if (it != resident_experts.end()) {
    // Cache HIT: Move to MRU position (back of the list)
    auto node = *(it->second);

    lru_expert_list.erase(it->second);

    lru_expert_list.push_back(node);

    it->second = std::prev(lru_expert_list.end());

    csv << layer_id << "," << expert_id
        << ",HIT," << resident_bytes << "\n";

    csv.flush();

    return true;
}
```

```C/C++
auto node = *(it->second);   //LRU Node
lru_expert_list.erase(it->second); //Remove from old location
lru_expert_list.push_back(node); //Add to new location
it->second = std::prev(lru_expert_list.end()); //Update Iterator
csv << layer_id << "," << expert_id
    << ",HIT," << resident_bytes << "\n";  //Store Hit in CSv
```

OFFCHIP DRAM Implementation

The implementation increases DDR5 “num_row” to simulate enough off chip DRAM capacity for MoE i.e. all expert weights can fit in properly.

```C/C++
//src/dram/memory_config.h
static MemoryConfig ddr5_32gb_x16_2ch = MemoryConfig(
    1,      // num_cube
    1,      // num_logic_cube
    2,      // num_channel
    2,      // num_rank
    4,      // num_bankgroup
    4,      // num_bank
    10485760, // num_row (2^17)*80 -> increased to simulate enough capacity for
MoE
```

```
    64,     // num_col (2^10 / 16)
    64,     // granul (bytes per request)
    MemoryConfig::MemoryType::DDR5 // mem_type
  );

//src/hardware/base.h
enum class MemoryTarget {
    HBM,
    OFFCHIP_DRAM
};
```

Offchip dram initialization (src/hardware/device.cpp)

```
C/C++
std::string offchip_cfg = config.offchip_dram_cfg_path;

if (offchip_cfg.empty()) {
    offchip_cfg = dram_cfg_path;
}

//Memory timing scale and memory organization initialization
double offchip_scale_factor = memory_scale_factor;

MemoryConfig offchip_memory_config = memory_config;

if (offchip_cfg.find("DDR5") != std::string::npos) {
    offchip_scale_factor = 0.625;
    offchip_memory_config = ddr5_32gb_x16_2ch;
}

//Second Interface and address mapper construction
offchip_dram_interface =
    DRAMInterface::Create(
        offchip_cfg,
        offchip_scale_factor
    );

offchip_mmap_controller =
    MMapController::Create(
        offchip_memory_config
    );
```

Assign Tensor Memory Objects (src/hardware/device.cpp)

```C/C++
void Device::setMemoryObject(Tensor::Ptr tensor) {
    if (tensor->weight_target == MemoryTarget::OFFCHIP_DRAM &&
        offchip_mmap_controller != nullptr) {

        offchip_mmap_controller->setMemoryObject(tensor);

    } else {

        mmap_controller->setMemoryObject(tensor);
    }
}
```

Offchip DRAM Request Submission (src/hardware/device.cpp)

```C/C++
void Device::run_ramulator(
    DRAMRequest_Ptr dram_request,
    MemoryTarget target
) {
    std::list<DRAMRequest::Ptr> request;
    request.push_back(dram_request);

    if (target == MemoryTarget::OFFCHIP_DRAM &&
        offchip_dram_interface != nullptr) {

        offchip_dram_interface->HandleRequest(request, 0);

    } else {

        dram_interface->HandleRequest(request, 0);
    }
}
void Device::run_ramulator_batch(
    const std::list<DRAMRequest_Ptr>& requests,
    MemoryTarget target
) {
    if (target == MemoryTarget::OFFCHIP_DRAM &&
        offchip_dram_interface != nullptr) {
        offchip_dram_interface->HandleRequest(requests, 0);
    } else {
        dram_interface->HandleRequest(requests, 0);
    }
}
```

Constructing DDR5 Requests (src/hardware/layer_impl.cpp)

```C/C++
ExecStatus issueRamulatorBatch(
    Device_Ptr device,
    LayerType layer_type,
    ProcessorType processor_type,
    DRAMRequestType dram_request_type,
    PIMOperandType pim_operand_type,
    const std::vector<Tensor_Ptr>& tensors,
    MemoryTarget target)
//Calculate aggregate size
long total_size = 0;
for (auto tensor : tensors) {
    total_size += tensor->getSize();
}
//Create execution key
CacheKey key = std::make_tuple(
    layer_type,
    processor_type,
    dram_request_type,
    total_size,
    target
);
//Bypass or not
bool bypass_cache =
    (target == MemoryTarget::OFFCHIP_DRAM);
```

Create the request list (src/hardware/layer_impl.cpp)

```C/C++
std::list<DRAMRequest::Ptr> requests;
for (auto tensor : tensors) {
    DRAMRequest::Ptr dram_request =
        DRAMRequest::Create(dram_request_type);
    if (target == MemoryTarget::OFFCHIP_DRAM &&
        device->offchip_mmap_controller) {
        auto offchip_mem_obj = MemoryObject::Create(
            tensor->getMMap(),
            tensor->getMemoryObject()->getLogicAddr(),
            tensor->getSize(),
            device->offchip_mmap_controller
        );
        dram_request->AddOperand(
            offchip_mem_obj,
            pim_operand_type
        );
```

```
    } else {
        dram_request->AddOperand(
            tensor->getMemoryObject(),
            pim_operand_type
        );
    }

    requests.push_back(dram_request);
}
```

Submit the Request List (src/hardware/layer_impl.cpp)

```C/C++
//Submit Request List
device->run_ramulator_batch(requests, target);

//Retrieve Execution Status from selected memory interface and return status
if (target == MemoryTarget::OFFCHIP_DRAM &&
    device->offchip_dram_interface) {

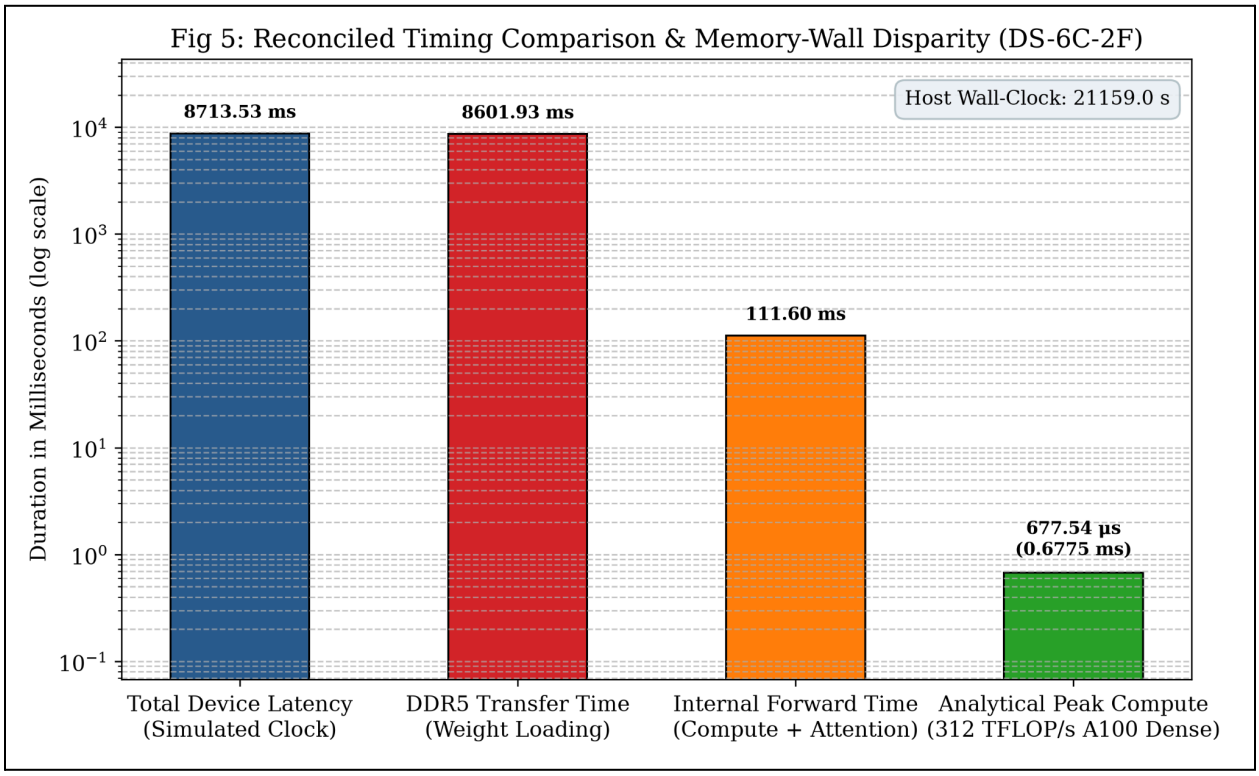
    exec_status =
        device->offchip_dram_interface->getExecStatus();

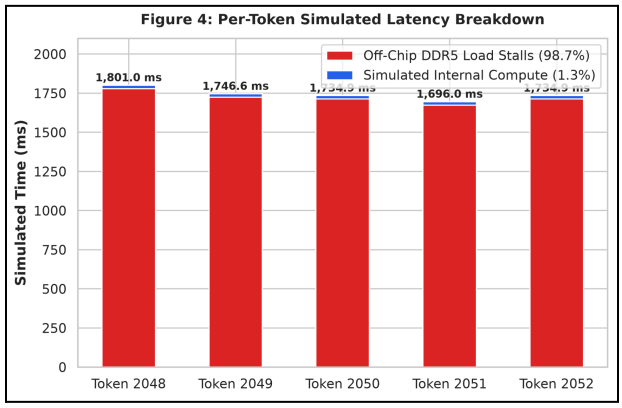
} else {

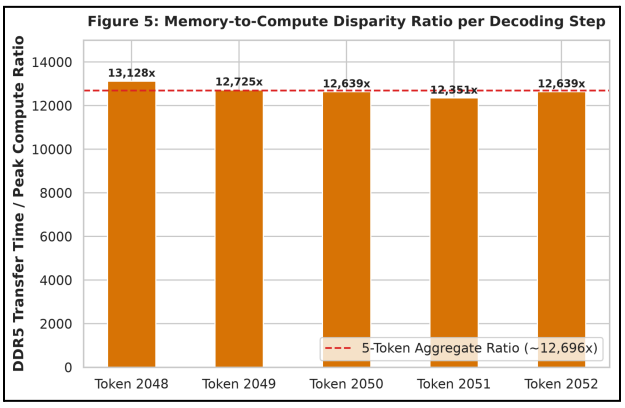
    exec_status =
        device->dram_interface->getExecStatus();
}

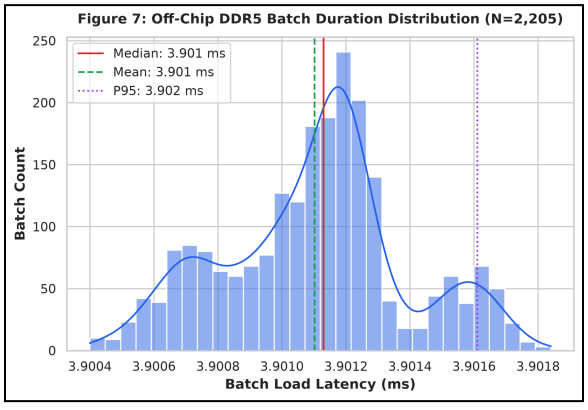
return exec_status;
```

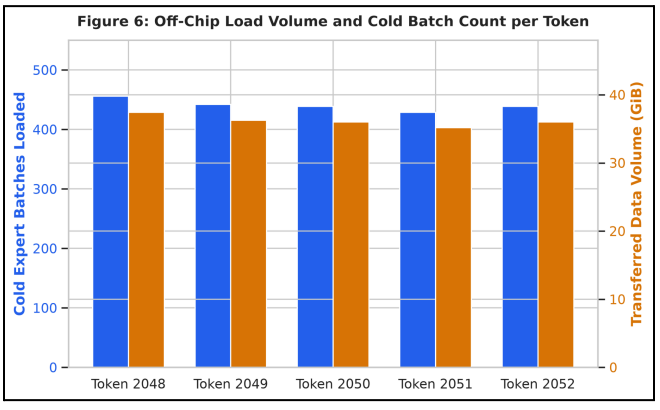
Off-Chip DRAM Loading vs Compute Time Plots & Analysis











Result Interpretation & Analysis

- **Plot - 1 : Overall Loading Time vs Compute Time**
  DDR5 loading takes approximately 77 times as long as internal forward computation in the overall timing comparison. This shows that evaluated configuration is loading-bound rather than compute-bound.

Metric	Duration	Interpretation
Total Simulated Device Latency	8713 ms	Total Modelled Device Simulation Time
DDR5 expert weight loading	8601 ms	Time attributed to loading/ transferring of expert weights
Internal Forward Computation	111 ms	Simulated Computation & Attention
Analytical Peak Compute Time	0.677 ms	Theoretical compute estimate at 312 TFLOPS/s
Host Wall Clock Time	21,159 s	Elapsed time reported for host-side execution

  1) Total Time = 8601 + 111 = 8713
  2) Load to Compute Ratio = 8601/111 = 77
  3) Percentage of Loading = 8601/8713 = 98.72%
  4) Percentage of Compute = 111/8713 = 1.28%

- **Plot - 2 : Per token simulated latency breakdown**
  - Measure the simulated latency of five consecutive decoding tokens
  - The loading component remains dominant throughout the sample.
  - Paper 3 addresses this issue by attempting to fetch likely future experts before they are needed. Its objective is not simply to reduce the total number of bytes transferred, but to overlap expert loading with useful computation so that less transfer time remains exposed on the critical path. The plot establishes the motivation for this approach

- **Plot - 3 : Memory to Compute Disparity Ratio**
  This plot shows the ratio between DDR5 transfer time and analytical peak- compute time for each decoding token. The values range from 12351 times to 13128 times. Hence it supports the conclusion that expert-weight movement is a major performance concern in the evaluated configuration.

- **Plot - 4 : Off Chip DDR5 Batch Duration Distribution**
  This histogram shows the distribution of loading latencies for 2,205 recorded DDR5 batches. It shows that the dominant loading cost is not necessarily caused by a few exceptionally slow batches. Instead, a large number of relatively similar loading operations can accumulate into a substantial total loading time.
  This finding is relevant to Paper 3 because APEX focuses on reducing the portion of expert-loading latency that delays computation. Predictable individual batch durations may help estimate transfer costs, but knowing the loading duration alone is not enough to hide it. The system must also identify experts early enough and have sufficient bandwidth and computation time available to overlap the transfers.

- **Plot - 5 : Off-Chip Load Volume and Cold-Batch Count per Token**
  This plot uses two vertical axes:
    - The blue bars show the number of cold expert batches loaded for each token.
    - The orange bars show the amount of data transferred, in GiB.

  A cold batch represents a batch classified by the experiment as requiring an off-chip load. The plot helps explain why loading dominates the timing breakdown. A large number of cold batches creates repeated opportunities for off-chip transfer latency to delay execution. The substantial volume of transferred data also places pressure on the available memory bandwidth.
  This is consistent with the motivation behind APEX. Paper 3 explains that expert parameters may reside off-chip because storing all experts in fast on-package memory is impractical under some capacity, cost, and power constraints. As a result, fetching the experts required for each token can become a major source of latency.

<u>Overall Interpretation/Analysis Summary</u>

The five plots consistently identify off-chip expert loading as the dominant modeled cost.
The overall timing plot reports 8,601 ms of DDR5 transfer time and 111 ms of internal forward time. Loading therefore accounts for approximately 98.72% of total simulated device latency and takes approximately 77.08 times as long as the internal forward computation.
The per-token breakdown shows that this imbalance persists across the five sampled decoding steps. The batch-duration distribution indicates that individual loading operations have a concentrated latency distribution, while the cold-batch and transferred-volume plot shows that each token is associated with hundreds of cold batches and a substantial volume of data movement.
Paper 3 provides the architectural context for these results: when expert weights reside off-chip, inference performance can be limited by the time required to fetch them. APEX addresses this problem by predicting expert demand and overlapping transfers with useful computation, reducing the portion of loading latency exposed on the critical path.
The main conclusion is that the evaluated configuration is loading-bound, and reducing exposed expert-loading latency is the most directly motivated optimization direction. The current plots

establish the bottleneck; additional controlled experiments are needed to quantify how much LRU residency, prefetching, or other memory-management strategies can reduce it.

Why Expert Loading Becomes a Performance Bottleneck in MoE Execution

- **Loading Perspective: Understanding the MoE Execution Flow**
  In a Mixture-of-Experts (MoE) model, each token activates only a small subset of the available experts through a routing mechanism. Although this reduces the amount of computation required compared with activating every expert, the weights of the selected experts must be available in GPU memory before their computations can proceed. In the off-chip DRAM configuration evaluated in this project, expert weights are stored in secondary memory, while GPU HBM acts as fast local memory. Consequently, when a selected expert is not resident in HBM, its weights must be transferred from off-chip DRAM before execution can continue. This dependency introduces memory-loading latency into the MoE execution flow and can leave the GPU waiting for data rather than performing useful computation.

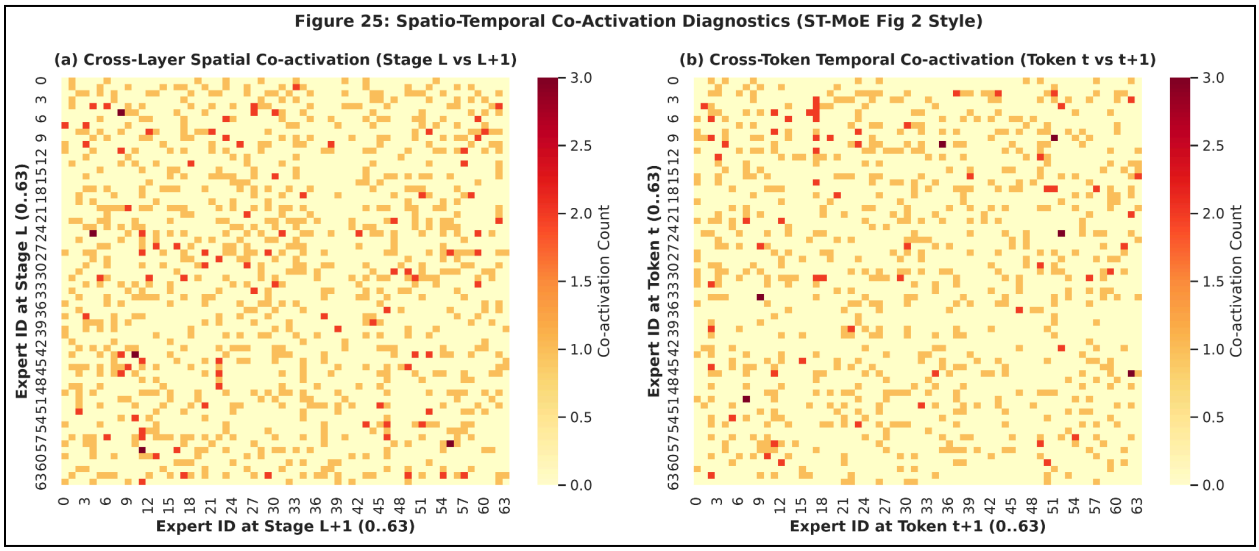
- **Code Based Perspective: How implementation produces loading latency**
  The system checks whether the selected expert's weights are resident in HBM. If they are available, the expert can proceed without another off-chip load. Otherwise, the system requests the required weights from secondary DRAM and manages HBM capacity using an LRU-based residency mechanism. When capacity is insufficient, previously resident weights may need to be evicted to accommodate the newly loaded expert. Although LRU can reduce repeated transfers when experts are reused while resident, it cannot eliminate the loading cost for experts that are absent from HBM. The performance impact depends on expert size, effective memory bandwidth, the number of cold loads, and the amount of reuse available in the workload.
- **Results: Findings of Real Experiment/Simulation with Off chip DRAM**
  The experimental results demonstrate the magnitude of this bottleneck. The overall timing plot reports 8,601.93 ms of DDR5 expert-weight loading compared with 111.60 ms of internal forward computation. Loading therefore takes approximately 77.1 times as long as computation and accounts for approximately 98.72% of the total simulated device latency. The per-token latency plot reinforces this observation, showing that off-chip loading stalls remain dominant across five consecutive decoding tokens, with the figure reporting approximately 98.7% of latency attributable to loading and 1.3% to internal computation. Furthermore, the batch-loading distribution reports a median latency of 3.901 ms across 2,205 batches, while the cold-batch and transferred-volume plot shows approximately 428–455 cold batches and 35–37 GiB of transferred data per token. Together, these results suggest that the cumulative cost of repeatedly loading substantial amounts of expert data, rather than the duration of an individual batch alone, is a major contributor to overall latency.

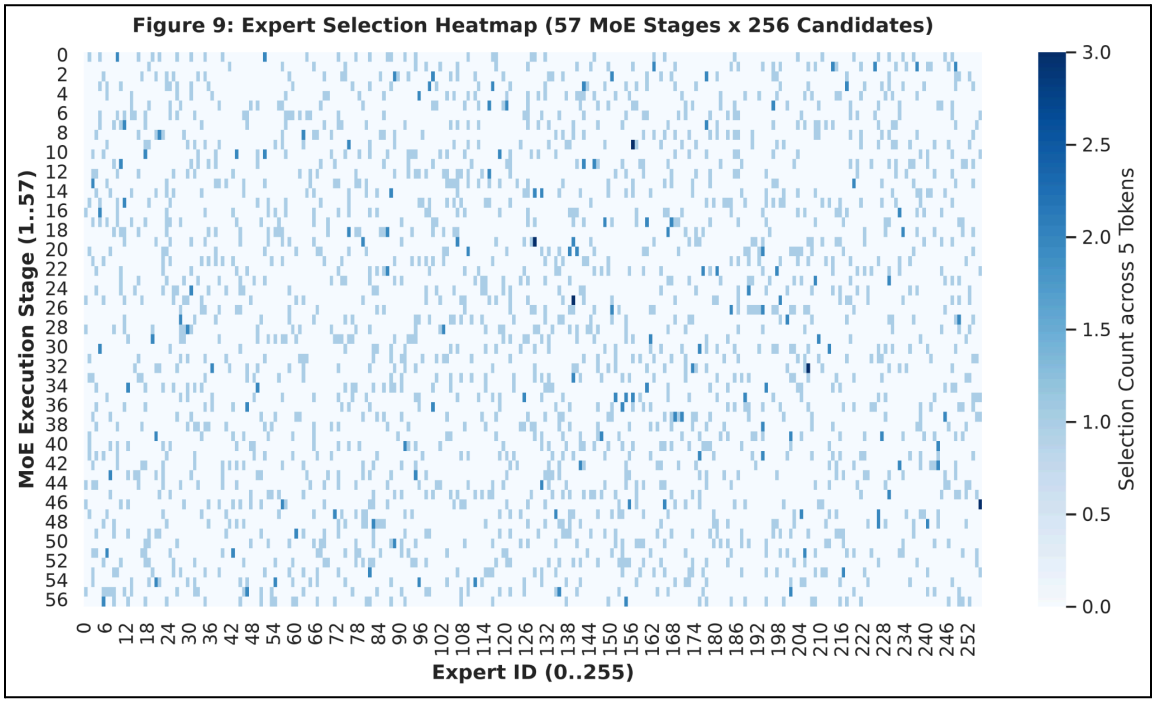
- In summary, expert loading becomes a performance bottleneck because MoE computation depends on the timely availability of selected expert weights, while transferring those weights from off-chip DRAM can take substantially longer than executing the experts themselves.

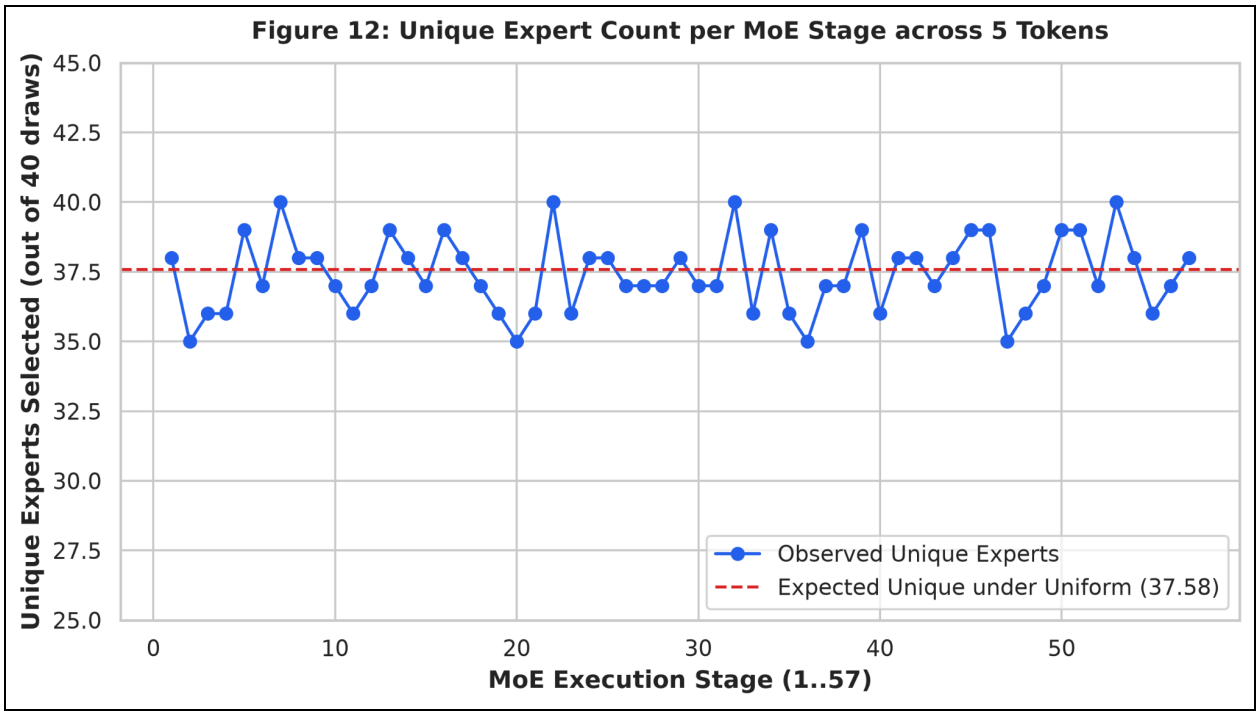
Temporal-Spatial Correlation in Expert Accesses

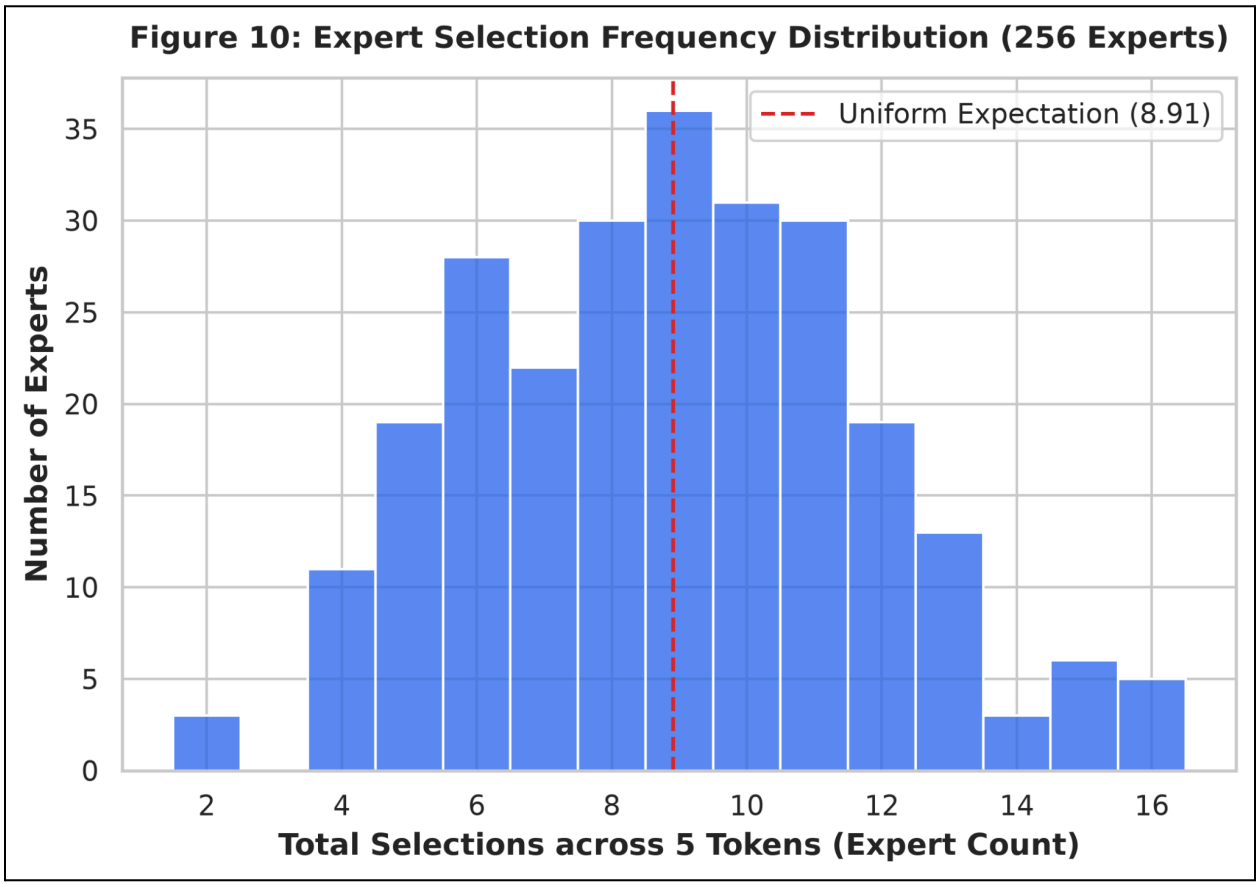
One potential way to reduce this cost is to exploit temporal and spatial correlations in expert accesses.

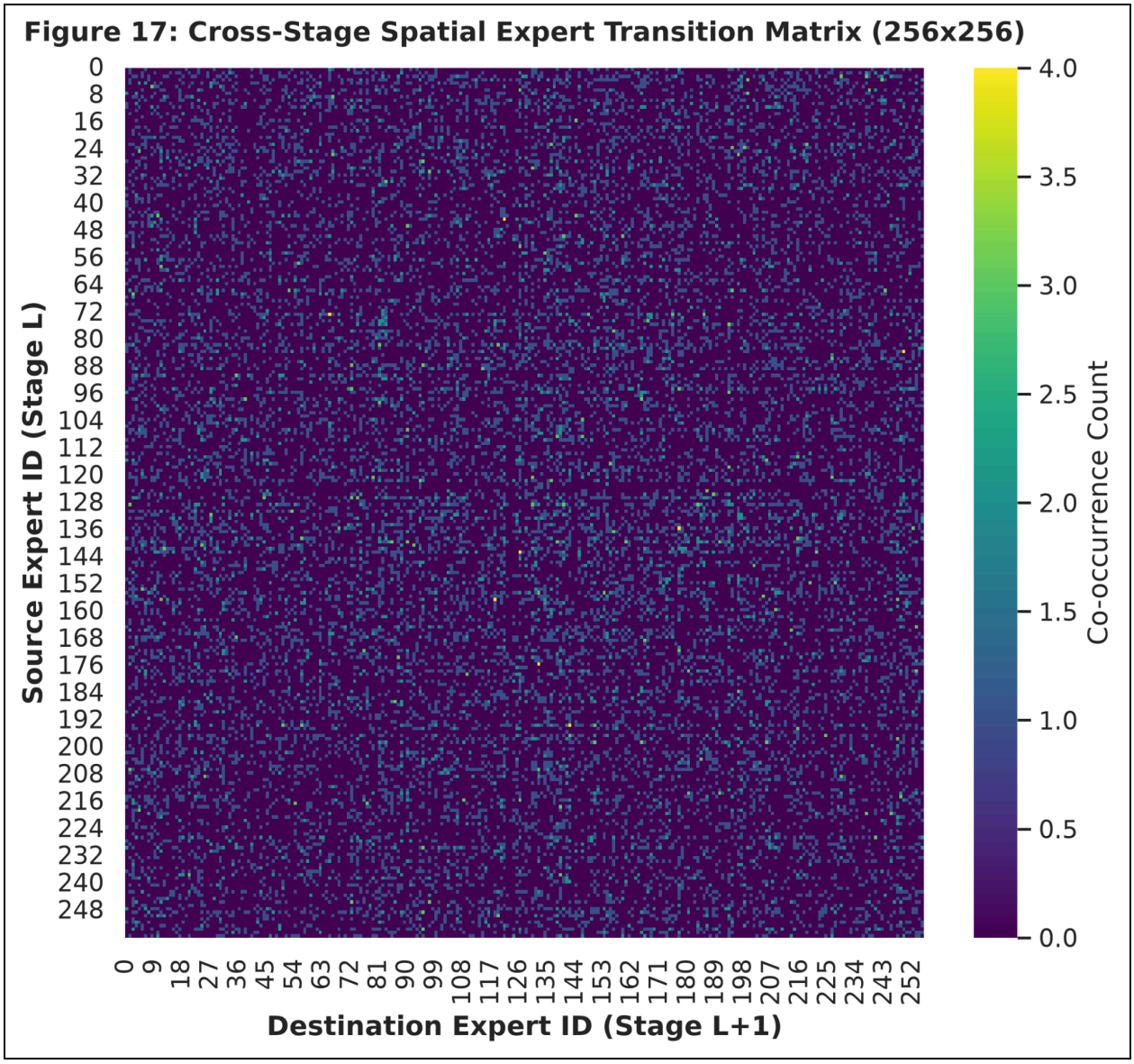
- Temporal correlation refers to whether experts selected for one token are likely to be selected again for the next token.
- Spatial correlation refers to whether selecting an expert in one MoE layer provides information about which experts are likely to be selected in a neighboring layer.






Figure 12: Unique Expert Count per MoE Stage across 5 Tokens
Unique Experts Selected (out of 40 draws)
45.0
42.5
40.0
37.5
35.0
32.5
30.0
27.5
25.0
0
10
20
30
40
50
MoE Execution Stage (1..57)
Observed Unique Experts
Expected Unique under Uniform (37.58)


Figure 10: Expert Selection Frequency Distribution (256 Experts)
Number of Experts
35
30
25
20
15
10
5
0
2
4
6
8
10
12
14
16
Total Selections across 5 Tokens (Expert Count)
Uniform Expectation (8.91)



Paper 2, which investigates spatio-temporal expert prefetching (ST-MoE), uses these temporal-spatial relationships to motivate predicting and prefetching experts before they are needed. If future expert accesses can be predicted accurately, their weights may be loaded in advance, potentially reducing stalls during inference.

- **<u>Plot - 1: Spatio-Temporal Co-Activation Diagnostics</u>**
  In both heatmaps, brighter cells represent higher co-activation counts, while lighter cells represent lower counts.
  The observed patterns appear sparse, with most cells showing low counts and a smaller number showing stronger co-activation. This suggests that some expert pairs occur together more frequently than others in the observed trace.

For the spatial heatmap, stronger cells may indicate that particular experts at stage (L) are associated with particular experts at stage (L+1).

For the temporal heatmap, stronger cells may indicate recurring relationships between experts selected for consecutive tokens. Such relationships could potentially help identify experts that are likely to be reused or accessed soon.

- Cross-layer spatial co-activation - Stage L vs. L+1
  - The vertical axis represents expert IDs at stage (L).
  - The horizontal axis represents expert IDs at the next stage, (L+1).
  - Each cell represents the observed co-activation count for a pair of experts across neighboring stages.
- Cross-token temporal co-activation - Token t vs. t+1
  - The vertical axis represents expert IDs selected for token (t).
  - The horizontal axis represents expert IDs selected for token (t+1).
  - Each cell represents the observed co-activation count across consecutive tokens.

- **Plot 2: Expert Selection Heatmap - 57 MoE Stages × 256 Candidates**

  This heatmap displays expert selections across 57 MoE execution stages and 256 candidate experts. The purpose is to visualize how expert selections are distributed across stages and whether some experts are selected repeatedly.
  - The vertical axis represents MoE execution stages, from 1 to 57.
  - The horizontal axis represents expert IDs, from 0 to 255.
  - The color intensity represents the number of selections observed across five tokens.

  The heatmap shows sparse selections across the expert dimension: most expert-stage combinations have low counts, while selected cells show higher activity.
  This pattern is consistent with sparse MoE routing, in which only a subset of the available experts is activated at a given stage.
- **Plot 3: Unique Expert Count per MoE Stage Across Five Tokens**
  - This line plot shows the number of distinct experts selected at each of the 57 MoE stages across five tokens.
  - The horizontal axis represents the MoE execution stage, while the vertical axis shows the number of unique experts selected.
  - The dashed red line represents the expected unique-expert count under the plot's uniform-selection reference, approximately 37.58.
- **Plot 4: Expert Selection Frequency Distribution - 256 Experts**
  - This histogram shows the total number of times each of the 256 experts was selected across five tokens.
  - The horizontal axis represents the total selection count for an expert, while the vertical axis represents the number of experts with that count.

    - The dashed red line marks the uniform-selection expectation of approximately 8.91 selections per expert.
- **Plot 5: Cross-Stage Spatial Expert Transition Matrix - 256 × 256**
    - This heatmap represents the observed co-occurrence counts between expert IDs at stage (L) and expert IDs at stage (L+1).
    - The vertical axis represents the source expert at stage (L).
    - The horizontal axis represents the destination expert at stage (L+1).
    - Each cell records how often the corresponding source and destination experts co-occurred in the observed trace.
    - Unlike the first plot's smaller spatial heatmap, this figure covers the full 256 × 256 expert-ID space.

Overall Analysis Summary

Taken together, the plots provide an exploratory characterization of expert-access behavior. They show how expert selections are distributed, how frequently experts occur, and which expert pairs co-occur across stages or tokens.

The strongest evidence of potential spatial and temporal relationships comes from the two co-activation plots and the cross-stage transition matrix. The frequency distribution and unique-expert-count plot provide supporting context about expert diversity and selection behavior.

The five plots build upon Paper 2 (ST-MoE) by examining spatial correlations between experts in adjacent MoE stages and temporal correlations across consecutive tokens. The expert-selection heatmap, unique-expert counts, and frequency distribution characterize routing sparsity, expert diversity, and selection popularity, while the co-activation heatmaps and transition matrix reveal potential relationships between expert accesses.

These observations motivate a spatio-temporal expert prefetcher that predicts future expert requests and loads their weights in advance. This aligns with Paper 2's prediction-guided prefetching and Paper 3's adaptive prefetching strategy, which aims to reduce exposed expert-loading latency by overlapping transfers with computation

References

[1] S. Yun, K. Kyung, J. Cho, J. Choi, J. Kim, B. Kim, S. Lee, K. Sohn, and J. H. Ahn, “Duplex: A Device for Large Language Models with Mixture of Experts, Grouped Query Attention, and Continuous Batching,” 2024 57th IEEE/ACM International Symposium on Microarchitecture (MICRO), 2024, pp. 1429–1443, doi: 10.1109/MICRO61859.2024.00105.

[2] Y. Zhao, R. C. Bunescu, A. Louri, A. Karanth, and K. Wang, “A Spatio-Temporal Expert Prefetching Framework for Efficient MoE-based LLM Inference,” arXiv preprint arXiv:2606.15453, 2026.

[3] A. Kanani, L. Badawi, and U. Y. Ogras, “APEX: Adaptive Expert Prefetching for Memory-Efficient Edge MoE Inference,” arXiv preprint arXiv:2608.11688, 2026; accepted at IEEE/ACM CODES 2026.